

**Software Technology 2
(7170)**

**Week 4 Tutorial & Computer Lab
Assignment 1: Individual Project**

Part 1: Simple Data Structures and Algorithms in Python

- This lab activity is assessed and contributes to the Assignment 1 (Individual Project) mark.
- Please go through Week 3,4 lecture recordings and Week_4_Tutorial_and Computer_Laboratory.doc available after Module 4 in Canvas (Assignment 1/Individual Project preparation activities)
- For assessment of this activity, you need to complete the following in week 4 tutorial/lab:
 - 1) attend your allocated computer lab class in Week 4,
 - 2) complete all tasks listed in this handout,
 - 3) demonstrate the working activities to your tutor in class, and
 - 4) Save your python project to a zip file and upload this zip file to **In-class Lab Work Demo Week 4** and upload it to your GitHub Repository. Make sure you include a commit message with date and time, when you upload this zip file in GitHub.
- Your mark is 0 if you do not demonstrate your project.
- You can leave the class after you complete the 4 requirements above.

Tasks

- Understanding Simple Data Structures in Python (text based RPG game in Week 4 lab activities folder)
- Understanding Simple Search Algorithms (Linear and Binary Search) and measuring their computational efficiency using Python Timeit module (Algorithms Project file in Week 4 lab activities folder)
- Understanding Simple Sorting Algorithms (Insertion sort and Bubble Sort) and measuring their computational efficiency using Python Timeit module (Algorithms Project file in Week 4 lab activities folder)

Download the Week_4_Lab_Activities.zip after Module 4 in Canvas, and unzip the folder. This folder content contains all the necessary activities needed for each task completion in this week.

Task 1: Understanding Simple Data Structures in Python

In this task, you will run the file **rpg.py** provided in the zip folder to understand how simple data structures such as lists, tuples, dictionaries and sets can be used to build a text based role playing game (rpg).

Try rpg.py and test the code for different scenarios for player winning the game, enemy winning the game and a result of draw, when player and enemy score is same. Show the screenshots for each successful test run below.

Player winning the game:

```
Player health: 85
Enemy health: 85

PLAYER TURN
Select move...
last stand
You selected: last stand
Player health: 70
Enemy health: 55

ENEMY TURN
Enemy selected: normal
Player health: 50
Enemy health: 55

PLAYER TURN
Select move...
last stand
You selected: last stand
Player health: 35
Enemy health: 25

ENEMY TURN
Enemy selected: last stand
Player health: 5
Enemy health: 10

PLAYER TURN
Select move...
normal
You selected: normal
Player health: 5
Enemy health: -10

GAME OVER: Player wins.

Process finished with exit code 0
```

Enemy winning the game:

```
last stand
You selected: last stand
Player health: 15
Enemy health: 10

ENEMY TURN
Enemy selected: special
Player health: 5
Enemy health: 15

PLAYER TURN
Select move...
heal
You selected: heal
Player health: 20
Enemy health: 15

ENEMY TURN
Enemy selected: special
Player health: 10
Enemy health: 20

PLAYER TURN
Select move...
special
You selected: special
Player health: 15
Enemy health: 10

ENEMY TURN
Enemy selected: normal
Player health: -5
Enemy health: 10

GAME OVER: Enemy wins.

Process finished with exit code 0
```

Result of draw:

```
Player health: 75
Enemy health: 75

PLAYER TURN
Select move...
heal
You selected: heal
Player health: 90
Enemy health: 75

ENEMY TURN
Enemy selected: last stand
Player health: 60
Enemy health: 60

PLAYER TURN
Select move...
last stand
You selected: last stand
Player health: 45
Enemy health: 30

ENEMY TURN
Enemy selected: last stand
Player health: 15
Enemy health: 15

PLAYER TURN
Select move...
last stand
You selected: last stand
Player health: 0
Enemy health: -15

DRAW.

Process finished with exit code 0
```

Task 2: Understanding Simple Search Algorithms and Measuring their Computational Efficiency in Python

In this task, you will learn about simple search algorithms such as **linear search** and **binary search** for working with simple data structures such as lists and dictionaries. You will also learn about an approach to measure algorithm efficiency using Python **Timeit** Module.

Open the **algorithms_project_file.py** from the unzipped folder, and run the code corresponding to:

- (1) Linear Search Algorithm and show the algorithm efficiency for different data sizes.
- (2) Binary Search Algorithm and show the algorithm efficiency for different data sizes.
- (3) Comparing linear and binary search efficiency for a sorted list generated in the code.

Show the results for each of the above test cases and comment on which algorithm performs better and why ?

Data Size- 1000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
-1
Linear search execution time: 0.001984 seconds
Binary search execution time: 0.000106 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.176935 seconds
Insertion Sort Execution Time: 1.438218 seconds
Binary Search (With Sorting) Execution Time: 1.377358 seconds
Linear Search Execution Time: 0.000128 seconds

Process finished with exit code 0
```

Data Size- 10000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
-1
Linear search execution time: 0.020721 seconds
Binary search execution time: 0.000144 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.163932 seconds
Insertion Sort Execution Time: 1.472038 seconds
Binary Search (With Sorting) Execution Time: 1.388764 seconds
Linear Search Execution Time: 0.000140 seconds

Process finished with exit code 0
```

Data Size- 50000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
-1
Linear search execution time: 0.104367 seconds
Binary search execution time: 0.000157 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.145502 seconds
Insertion Sort Execution Time: 1.420440 seconds
Binary Search (With Sorting) Execution Time: 1.402633 seconds
Linear Search Execution Time: 0.000151 seconds

Process finished with exit code 0
```

Comparison of 50000 data size:

Linear- 0.104367Sec

Binary- 0.000157 Sec

So, binary was faster than Linear

Comment: Binary search is faster than the linear search because it has a Divided and Conquerable strategy that will substantially reduce the number of elements which it has to search. Linear search examines each item in one by one, but binary search eliminates half of the information left after each comparison.

Task 3: Understanding Simple Sorting Algorithms and Measuring their Computational Efficiency in Python

In this task, you will learn about simple sorting algorithms such as **insertion sort** and **bubble sort** for working with simple data structures such as lists and dictionaries. You will also learn about an approach to measure algorithm efficiency using Python **Timeit** Module.

Open the **algorithms_project_file.py** from the unzipped folder, and run the code corresponding to:

- (4) Insertion Sort Algorithm and show the sorted list for different data sizes
- (5) Bubble Sort Algorithm and show the sorted list for different data sizes.
- (6) Measure and compare the runtimes of Bubble Sort and Insertion Sort.

Show the results for each of the above test cases and comment on which sorting algorithm performs better and why ?

Data Size- 1000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
-1
Linear search execution time: 0.001984 seconds
Binary search execution time: 0.000106 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.176935 seconds
Insertion Sort Execution Time: 1.438218 seconds
Binary Search (With Sorting) Execution Time: 1.377358 seconds
Linear Search Execution Time: 0.000128 seconds

Process finished with exit code 0
```

Data Size- 10000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
-1
Linear search execution time: 0.020721 seconds
Binary search execution time: 0.000144 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.163932 seconds
Insertion Sort Execution Time: 1.472038 seconds
Binary Search (With Sorting) Execution Time: 1.388764 seconds
Linear Search Execution Time: 0.000140 seconds

Process finished with exit code 0
```

Data Size- 50000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
-1
Linear search execution time: 0.104367 seconds
Binary search execution time: 0.000157 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.145502 seconds
Insertion Sort Execution Time: 1.420440 seconds
Binary Search (With Sorting) Execution Time: 1.402633 seconds
Linear Search Execution Time: 0.000151 seconds

Process finished with exit code 0
```

Comparison of 50000 data size:

Bubble Sort- 3.145502 Sec

Insertion Sort- 1.420440 Sec

Insertion Sort is faster than Bubble sort

Comment: The time of Insertion sort is lower than that of Bubble sort because it has fewer swaps and direct items into their corresponding positions, and is more efficient when dealing with data that is almost sorted.

Task 4: Challenge Task - Combining Sorting Algorithms with Search Algorithms

Now that you have learned two simple searching algorithms and two simple sorting algorithms, it is interesting and useful to understand what happens when you combine them.

The challenge is to:

1. Combine insertion sort with linear search and test the combination for different data sizes for their run times.
2. Combine Bubble sort with linear search and test the combination for different data sizes for their run times.
3. Combine Insertion sort with Binary search and test the combination for different data sizes for their run times.
4. Combine Bubble sort with Binary search and test the combination for different data sizes for their run times.

Compare the performance for each combination of sorting and searching algorithm in this task with unsorted search algorithms you have examined in Task 2.

Comment on which combination performs better and why?

Data Size- 1000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
8
Linear search execution time: 0.599055 seconds
Binary search execution time: 0.000245 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.115932 seconds
Insertion Sort Execution Time: 1.404650 seconds
Binary Search (With Sorting) Execution Time: 1.378038 seconds
Linear Search Execution Time: 0.000199 seconds
Insertion Sort + Linear Search : 0.013923 sec
Bubble Sort + Linear Search : 0.029515 sec
Insertion Sort + Binary Search : 0.013832 sec
Bubble Sort + Binary Search : 0.029279 sec
Unsorted Linear Search (baseline): 0.000018 sec

Process finished with exit code 0
```

Data Size- 10000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
8
Linear search execution time: 1.082719 seconds
Binary search execution time: 0.000194 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.071827 seconds
Insertion Sort Execution Time: 1.420928 seconds
Binary Search (With Sorting) Execution Time: 1.375476 seconds
Linear Search Execution Time: 0.000147 seconds
Insertion Sort + Linear Search : 1.457394 sec
Bubble Sort + Linear Search : 3.106749 sec
Insertion Sort + Binary Search : 1.390757 sec
Bubble Sort + Binary Search : 2.957681 sec
Unsorted Linear Search (baseline): 0.000238 sec

Process finished with exit code 0
```

Data Size- 50000

```
C:\Users\saumi\AppData\Local\Programs\Python\Python313\python.exe
-1
-1
1
8
Linear search execution time: 0.244667 seconds
Binary search execution time: 0.000185 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 3.122211 seconds
Insertion Sort Execution Time: 1.423717 seconds
Binary Search (With Sorting) Execution Time: 1.380030 seconds
Linear Search Execution Time: 0.000113 seconds
Insertion Sort + Linear Search : 45.740356 sec
Bubble Sort + Linear Search : 100.935096 sec
Insertion Sort + Binary Search : 42.745675 sec
Bubble Sort + Binary Search : 96.919387 sec
Unsorted Linear Search (baseline): 0.000908 sec

Process finished with exit code 0
```

Best in general: Unsorted Linear Search.

Combination of the best performances: Binary Search and Insertion Sort.

Worst: The combinations of Bubble Sort since it is not an efficient sorting method.

Comment: The most general approach is the fastest, unsorted linear search since it does not involve any sorting, which is extremely slow.

Out of the sorted methods, the Insertion Sort + Linear Search and the Insertion Sort + Binary Search are much more effective than the Bubble Sort combinations. This is because the Insertion Sort is more efficient than the Bubble Sort since it entails fewer swaps and operations.

Although Binary Search is very quicker than Linear Search, this does not benefit us since the time taken in sorting is more than the overall time taken.

- Please complete this report with your test results and your comments for each task and questions asked.
- Demonstrate/present the tasks to your tutor in class.
- Prepare a report, similar to this handout, but with results included for your tests for each task and your comments, and save the report as **Week_4_lab_report.pdf**.
- Include this report as well as python scripts for each task (task 1 to task 4) in a separate folder and zip it, with a title (**Week_4_LabActivities.zip**).
- Upload the zip file to your github repo, and add a commit message with date and time details.
- You can leave the lab now. Please note your mark is 0 if you do not present/demonstrate your working project in lab class. This must be done in **Week 4**.

Mark for this tutorial: 8 marks